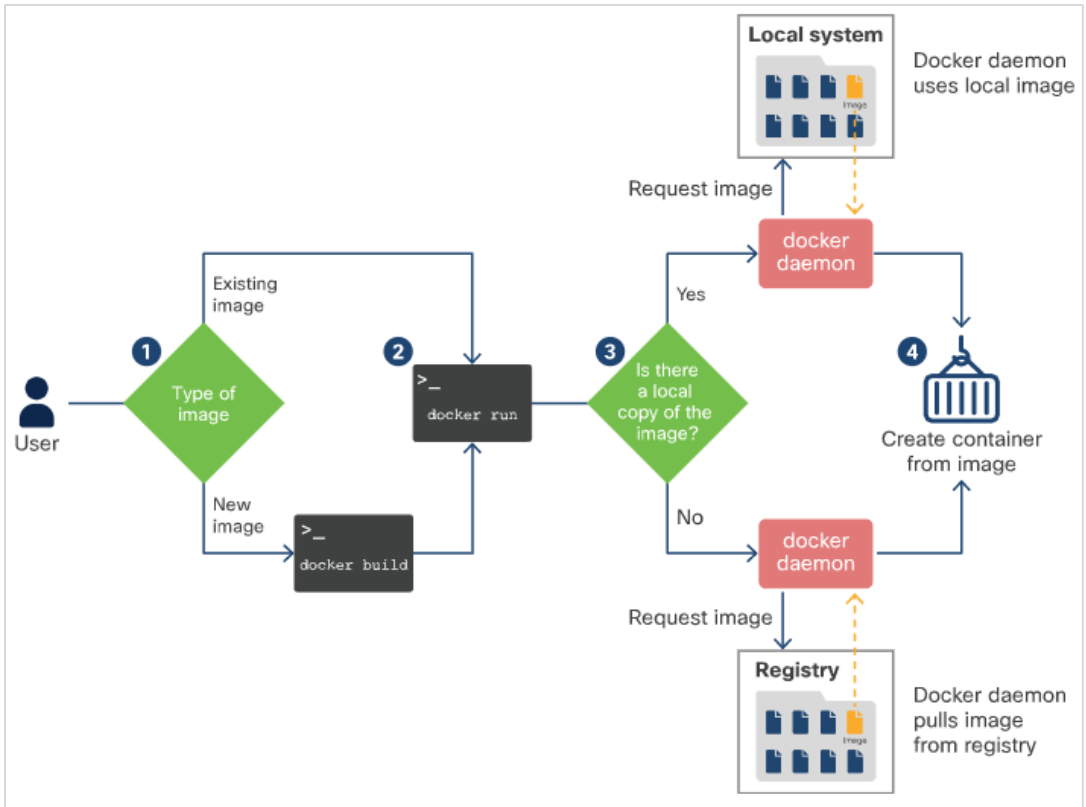
Docker:

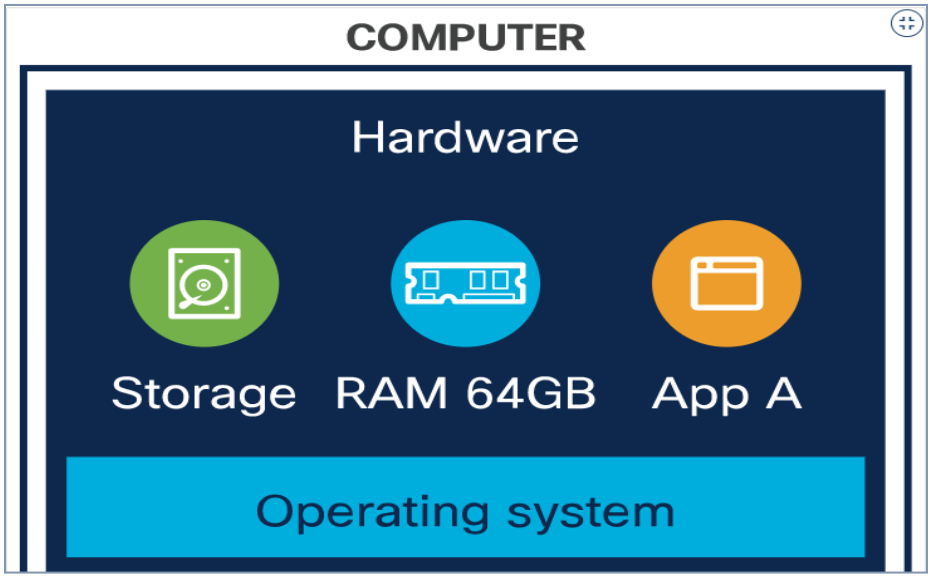
The most popular way to containerize an application is to deploy it as a Docker container. A container is a way of encapsulating everything you need to run your application, so that it can easily be deployed in a variety of environments. Docker is a way of creating and running that container. Docker is a format that wraps a number of different technologies to create containers. A Docker image is a set of read-only files that have no state and contains source code, libraries, and other dependencies needed to run an application. A Docker container is the run-time instance of a Docker image. Creating a container involves pulling an image or a template from a repository, then using it to create a container.

The workflow of creating a container is as follows:
Step 1: Either create a new image using docker build or pull a copy of an existing image from a registry using docker pull.
Step 2: Run a container based on the image using docker run or docker container create.
Step 3: The Docker daemon checks to see if it has a local copy of the image. If it does not, it pulls the image from the registry.
Step 4: The Docker daemon creates a container based on the image and, if docker run was used, logs into it and executes the requested command.



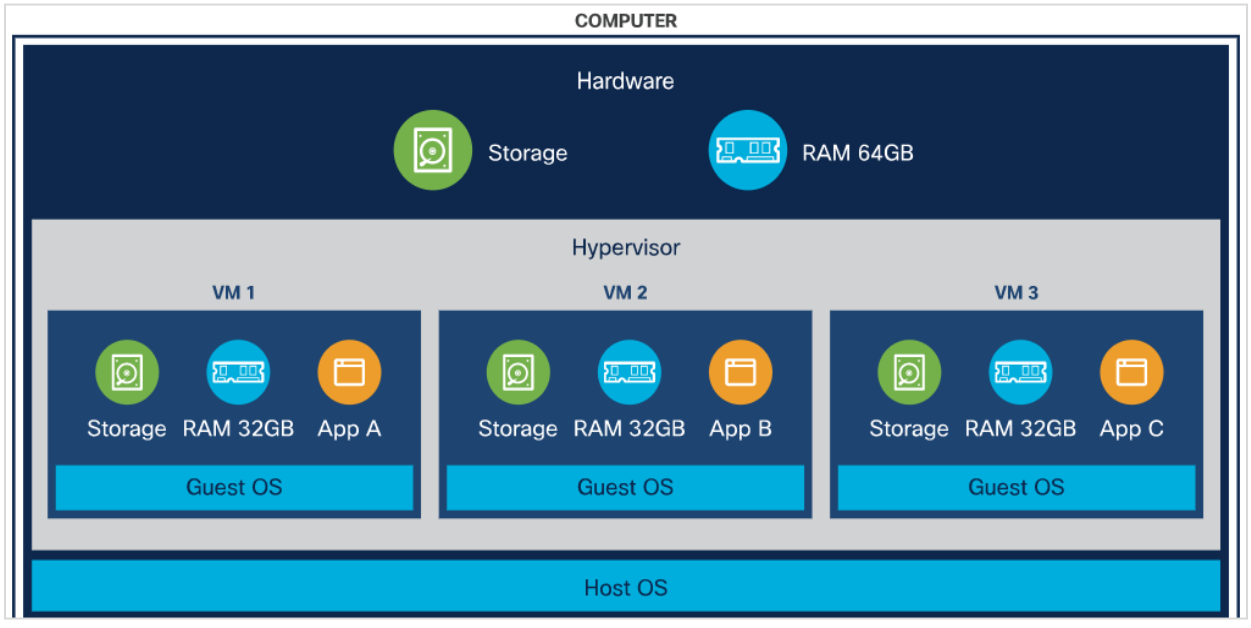
Bare Metal:

A bare metal deployment is essentially deploying to an actual computer. It is used to install a software directly on the target computer. In this method, software can directly access the operating system and the hardware. It is useful for situations requiring access to specialized hardware, or for High Performance Computing (HPC) applications. It is now used as infrastructure to host virtualization and cloud frameworks.



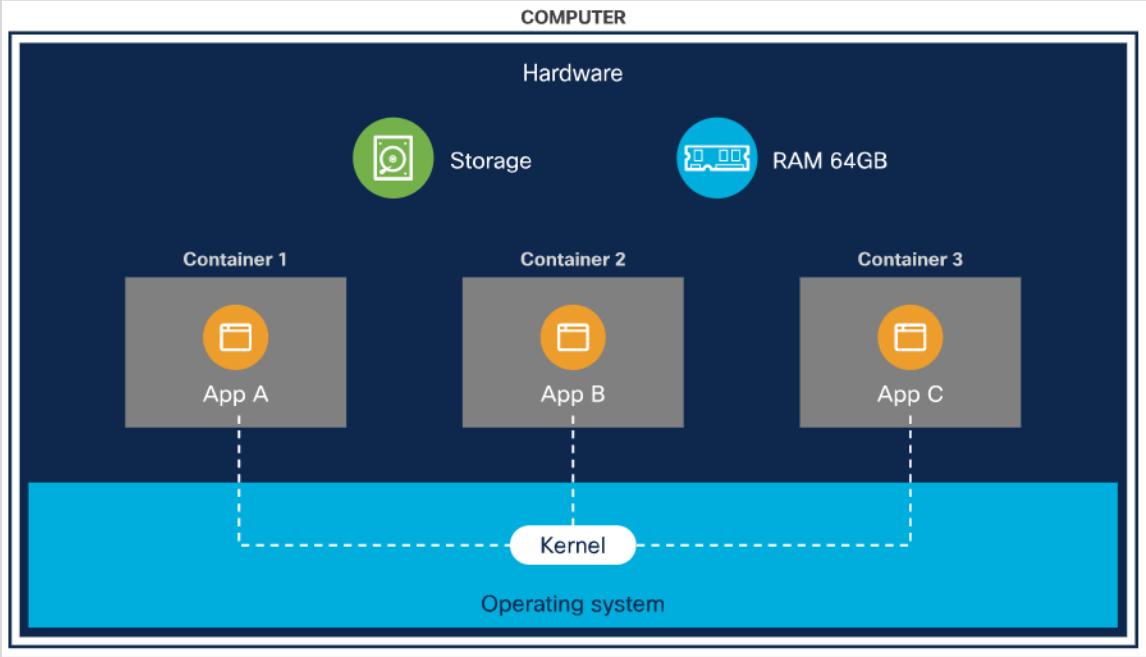
Virtual Machines (VMs):

Virtual machines share the resources of the host. It is like a computer within the computer and has its own computing power, network interfaces, and storage. Hypervisor is software that creates and manages VMs. VMs run on top of a hypervisor that provides VMs with simulated hardware, or with controlled access to underlying physical hardware.



Container-Based Infrastructure:

Containers were designed to provide the same benefits as VMs, such as workload isolation and the ability to run multiple workloads on a single machine but are designed to start up quickly. Containers share resources of the host including the kernel. A container shares the operating system of the host machine and uses container-specific binaries and libraries.



Container:

Docker Container is a standardized unit which can be created on the fly to deploy a particular application or environment. It could be an Ubuntu container, CentOS container, etc. to full-fill the requirement from an operating system point of view. Also, it could be an application-oriented container like CakePHP container or a Tomcat-Ubuntu container etc. Docker Containers are the ready applications created from Docker Images. Or you can say they are running instances of the Images and they hold the entire package needed to run the application. This happens to be the ultimate utility of the technology. Containers are a method of building, packaging and deploying software.

the developer will create a tomcat docker image (An Image is nothing but a blueprint to deploy multiple containers of the same configurations) using a base image like Ubuntu, which is already existing in Docker Hub (the Hub has some base images available for free) . Now this image can be used by the developer, the tester and the system admin to deploy the tomcat environment. Docker Container clearly wins again from Virtual Machine based on Startup parameter.

Container Orchestration:

Container orchestration is the automation of all aspects of coordinating and managing containers. Container orchestration is focused on managing the life cycle of containers and their dynamic environments. Container orchestration is all about managing the lifecycles of containers, especially in large, dynamic environments. Kubernetes is an open-source container management (orchestration) tool. It's container management responsibilities include container deployment, scaling & descaling of containers & container load balancing. Container orchestration automates the deployment, management, scaling, and networking of containers. Software teams use container orchestration to control and automate many tasks:

- Configuring and scheduling of containers.
- Provisioning and deployment of containers.
- Redundancy and availability of containers.
- Availability of containers.
- Scaling up containers to spread application load evenly across host infrastructure.
- Movement of containers from one host to another if there is a shortage of resources.
- Allocation of resources between containers.
- External exposure of services running in a container with the outside world.
- Load balancing of service discovery between containers.
- Health monitoring of containers and hosts.
- Health monitoring of containers.
- Securing the interactions between containers.
- Configuration of an application in relation to the containers running it.